

Revision 01-02 · What Is Inference Engineering + The Inference Stack

Interactive revision of [lesson 01](#) and [lesson 02](#): each question as asked, how I reasoned it out (my words, written in full), where I slipped, and the one-line takeaway.

Q1 · Can one user get 300 tokens/s? SOLID

Covers: 01, 16

QUESTION

We serve Llama 3 8B in BF16 on one H100, with only one user online. The PM wants to advertise "300 tokens/s streaming". Can we promise it?

How I solved it

1. The model has about 8 billion parameters, and BF16 stores each in 2 bytes, so the weights are about **16 GB**.
2. An H100's memory bandwidth is about **3.35 TB/s**.
3. Every decode step reads all the weights once: $16 \times 10^9 \text{ bytes} \div 3.35 \times 10^{12} \text{ bytes/s} = (16 \div 3.35) \times 10^{-3} \text{ s} \approx \textbf{4.8 ms}.$
4. In one second that's $1 \div 4.8 \text{ ms} \approx 200$ steps, one token each: about **200 tokens/s at most**.
5. 300 is above that ceiling, so the answer is **no**.

What to add next time

- Name the limiting resource out loud: **memory bandwidth**, not compute. The math units are mostly idle.
- 200 tokens/s is a **ceiling**, not the expected speed. Real batch-1 runs land 3–4× slower (lesson 16), and the ceiling drops as the KV cache grows.
- To reach 300, read fewer bytes: 8-bit weights are 8 GB $\rightarrow \sim 2.4 \text{ ms} \rightarrow$ a ceiling of ~ 415 tokens/s.

Q2 · 64 users at once: step time and speed PARTIAL

Covers: 01, 16

QUESTION

Same GPU and model, but 64 users are chatting at once, each with about 4,096 tokens of context in their KV cache (128 KB per token). How long is one decode step, what is the total tokens/s, and what does each user get?

How I solved it (with hints)

1. I couldn't recall the formula, so I went back to the principle from Q1: **time per step $\approx$ bytes read in that step $\div$ bandwidth**. The only real question is which bytes get read.
2. **Weights**: batched, so the 16 GB is read **once per step for everyone**, not once per user.
3. **KV cache**: each user's new token attends to that user's own past tokens, so every user's whole cache is read: $4,096 \times 128 \text{ KB} \approx 0.5 \text{ GB}$ each, $\times 64 \approx$ **32 GB** (about 34 GB in decimal units).
4. **Total read per step**: $16 + 32 \approx 48 \text{ GB}$, $\div 3.35 \text{ TB/s} \approx$ **14 ms** (the lessons get 15 ms with exact sizes).
5. **Per user**: $14 \text{ ms} = 0.014 \text{ s}$, so $1 \div 0.014 \approx$ **71 tokens/s**.
6. **Total**: each step gives all 64 users a token, so total = batch $\times$ (1 $\div$ step time) = batch $\div$ step time: $64 \div 0.014 \text{ s} = 71 \times 64 \approx$ **4,500 tokens/s**.
7. **Versus batch 1**: the GPU makes $4,500 \div 200 \approx$ **22 $\times$ more**, while each user slows from 200 to 71, only **$\sim 2.8\times$ slower**.

The 16 GB weight read is shared by everyone; only the KV cache is per user. The price of batching is KV memory (32 GB here), growing with batch $\times$ context.

RE-DRILL

Per-request KV = tokens in the conversation $\times$ 128 KB. Always sanity-check ms $\rightarrow$ tokens/s against batch 1 (4.8 ms $\rightarrow$ 200).

Q3 · Cost per million tokens **PARTIAL**

Covers: 01

QUESTION

The H100 rents for \$3/hour. What does 1 million output tokens cost at ~ 200 tokens/s (one user), and at $\sim 4,500$ tokens/s (64 users batched)?

Step-by-step answer. Cost = price per hour $\times$ hours of GPU needed.

Step	One user (200 tok/s)	64 users (4,500 tok/s)
Time for 1M tokens	$1,000,000 \div 200 =$ 5,000 s	$1,000,000 \div 4,500 \approx$ 222 s
Convert to hours	$5,000 \div 3,600 \approx$ 1.39 h	$222 \div 3,600 \approx$ 0.06 h
Cost = \$3/h $\times$ hours	\$4.17	\$0.18

Ratio: $4.17 \div 0.18 \approx$ **22 $\times$ cheaper**, the same factor as the throughput gain, because the GPU's hourly price doesn't change.

RE-DRILL

Write units next to every number (s, h, \$/h) so an upside-down formula shows itself. Sanity check: "about an hour, so about \$3?"

Q4 · Why is prefill of 500 tokens only ~2× one decode token? **PARTIAL**

Covers: 01, 15

QUESTION

Prefill pushes a 500-token prompt through in ~8 ms; one decode token takes ~4.8 ms. Why so close? Which resource limits each phase?

My explanation (concept, correct). In prefill all the prompt tokens are available up front, so nothing waits and the work runs in parallel: the compute units stay busy, so prefill is compute-intensive. In decode only one token at a time flows through all the weights, so the GPU spends most of its time waiting for data: memory-intensive.

Gap: I couldn't show *with numbers* that decode is memory-bound.

Step-by-step proof: compare math time with byte time; the bigger one sets the speed.

- Facts:** each parameter does 2 FLOPs per token (multiply + add). H100 $\approx$ 989 TFLOPS and 3.35 TB/s.
- Decode math:** $2 \times 8\text{B} = 16 \times 10^9$ FLOPs. GPU: 989 TFLOPS = 989×10^{12} FLOP/s $\approx 1,000 \times 10^{12} = 10^{15}$ FLOP/s. Divide: $16 \times 10^9 \div 10^{15} = 16 \times 10^{-6} \text{ s} = \mathbf{16 \mu\text{s}}$.
- Decode bytes:** 16×10^9 bytes $\div 3.35 \times 10^{12}$ bytes/s = $(16 \div 3.35) \times 10^{-3} \text{ s} = \mathbf{4.8 \text{ ms}}$ ($3.35 \times 5 \approx 16.75$, so $16 \div 3.35 \approx 4.8$).
- $4.8 \text{ ms} \div 16 \mu\text{s} \approx \mathbf{300\times}$: decode spends ~300× more time waiting for bytes than doing math. Math units idle >99%: **memory-bound**.
- Ridge point:** 989×10^{12} FLOP/s $\div 3.35 \times 10^{12}$ B/s; tera and seconds cancel $\rightarrow 989 \div 3.35 \approx \mathbf{295 \text{ FLOP/byte}}$ ($3.35 \times 300 \approx 1,000$). Batch-1 decode does ~1.
- Prefill 500 tokens:** weights still read once (4.8 ms), but math = $500 \times 16 \mu\text{s} = \mathbf{8 \text{ ms}}$. Math > bytes: **compute-bound**, $\approx 8 \text{ ms}$.

RE-DRILL

Powers-of-ten division: giga $\div$ tera = 10^{-3} , so $16 \text{ G} \div 1,000 \text{ T} = 16 \mu\text{s}$. Say "max(math, bytes)" out loud.

Q5 · At what batch does decode become compute-bound? **SOLID**

Covers: 01

QUESTION

With zero context, raise the batch size. At what batch does decode flip from memory-bound to compute-bound, and which number does that match?

Answer: around **batch 300**, the ridge point. $\text{Ridge} = 989 \div 3.35 \approx 295$, so batch 295 is still memory-bound and 296 is compute-bound.

Why batch = ridge: in BF16 one step does $2 \times P \times B$ FLOPs and reads $2 \times P$ bytes of weights, so FLOPs per byte = **B**. Decode turns compute-bound once B passes the ridge.

Caveat: with real context every user adds KV bytes too, so the flip comes much later or never: batch 296 at 4k context is still memory-bound.

Q6 · Autoscaling on the wrong signal PARTIAL

Covers: 02, 18

QUESTION

Four replicas of Llama 3 8B sit behind an autoscaler that adds a replica when CPU utilization passes 70%. During a spike, users wait 5+ seconds for the first token, but CPU shows 12% on every server, so nothing scales. What's wrong, and what should the autoscaler watch instead?

Answer

- **CPU is the wrong signal.** The GPU does the work, so the CPU sits mostly idle even when a replica is overloaded.
- **Watch two engine signals instead:** 1. **Queue length** (requests waiting, or time spent waiting): the 5 seconds is mostly time in the engine's queue, because prefill itself takes milliseconds. 2. **KV cache usage:** each user holds its own cache, so a full cache means no room for more users. Use the engine's metric, not raw GPU memory: engines reserve ~90% of GPU memory at startup, so `nvidia-smi` always looks "full".
- **Scale early, on the trend:** a new replica must load 16 GB of weights before it serves anyone.

RE-DRILL

When a latency number looks huge, ask where the time goes: compute (prefill/decode) or waiting (queue).

Q7 · How many users fit on one GPU? SOLID

Covers: 02, 18, 19

QUESTION

One H100 (80 GB) serves Llama 3 8B in BF16 (16 GB of weights). Ignoring runtime overhead, how many tokens of KV cache fit (128 KB per token), and how many 4,096-token conversations can run at once?

How I solved it

1. Memory left for the cache: $80 - 16 = 64 \text{ GB}$ (the weights take 16).
2. Tokens: $64 \times 10^9 \text{ bytes} \div 128 \times 10^3 \text{ bytes} = 0.5 \times 10^6 \approx 500\text{k tokens}$ (488k with 128 KB = 131,072 bytes).
3. Users: $500 \times 10^3 \div 4,096 \approx 122 \text{ users}$ (~ 119 with the exact numbers).

Caveat: this is the naive upper bound. Runtime, activations and headroom take $\sim 11 \text{ GB}$ more, so a real engine gets $\sim 400\text{k tokens}$ (lesson 18).

Q8 · Kernel launch overhead **PARTIAL**

Covers: 02, 17, 18

QUESTION

One decode step launches ~ 300 kernels at $\sim 4 \mu\text{s}$ of CPU time each; the ideal step (bytes $\div$ bandwidth) is 4.8 ms. How much time goes to launches, as a share of the step, and how do you cut it?

Answer

- Launch time: $300 \times 4 \mu\text{s} = 1,200 \mu\text{s} = 1.2 \text{ ms}$, which is $1.2 \div 4.8 = 25\%$ of the ideal step.
- **Fix 1, kernel fusion:** merge several kernels into one, so there are fewer launches and fewer round trips to memory.
- **Fix 2, CUDA graphs:** record the step's launches once and replay them with a single launch, so ~ 300 CPU launches become ~ 1 . Needs fixed tensor shapes (lesson 17: preallocated KV cache) and costs 1–5 GB of memory (lesson 18).

RE-DRILL

Percentages: divide the part by the whole, then sanity-check with a simple fraction ($\frac{1}{4}$, $\frac{1}{3}$, $\frac{1}{2}$). Remember the pair "fusion + CUDA graphs".

Q9 · Cold start of a new replica **SOLID**

Covers: 02

QUESTION

During a spike the autoscaler launches a new replica, which must load 16 GB of weights from storage at ~ 2 GB/s. How long until it serves, what does that mean for when to trigger, and how can you shorten it?

Answer

- **Load time:** $16 \text{ GB} \div 2 \text{ GB/s} = 8 \text{ s}$ of weight loading alone. Add container start, GPU init and CUDA graph capture, and real cold starts are often tens of seconds.
- **When to trigger:** early, on a rising trend, e.g. **KV cache usage at 70–80%** or **queue length growing**, so the new replica is ready before the current ones saturate.
- **How to shorten it:**
 - Faster source: keep weights in **host RAM**, or on a **local NVMe cache** instead of network storage.
 - Fewer bytes: **FP8 weights** (8 GB) halve the load to ~ 4 s.
 - Don't load at spike time: keep a **warm standby pool** of replicas already loaded.

RE-DRILL

For any "time = bytes $\div$ speed" problem, the levers are always: fewer bytes, faster speed, or do it ahead of time.

Exercise · Round-robin vs prefix-aware router SOLID

Covers: 02

QUESTION

12 requests, each a shared system prompt (A, B, C cycling, 1,000 tokens) plus a fresh 50-token user message, go to 4 replicas that cache prompts they've prefilled. How many prompt tokens does the fleet prefill under round-robin ($i \% 4$) vs prefix-aware routing ($\text{prefix_id} \% 4$)?

Setup I needed clarified first: `policy` exists to compare two routers with one function; `prefix_id` is which system prompt (not which replica); under round-robin the cycles $i \% 4$ and $i \% 3$ don't line up, so every replica sees A, B and C and caches nothing useful.

The structure:

1. **State:** one set per replica, holding the prompt ids it has cached.
2. **Pick the replica:** round-robin $i \% n$; prefix-aware $\text{prefix_id} \% n$.
3. **Cost:** `user_len` always, plus `prefix_len` only if that replica hasn't cached the prompt.
4. **Update:** add the prompt to that replica's set.
5. **Output:** sum of costs.

```
def total_prefill_tokens(requests, n_replicas, policy):
    cached = [set() for _ in range(n_replicas)] # Step 1 · state per replica
    total = 0
    for i, (prefix_id, prefix_len, user_len) in enumerate(requests):
        if policy == "round_robin": # Step 2 · pick the replica
            replica = i % n_replicas
        else:
            replica = prefix_id % n_replicas
        cost = user_len + (0 if prefix_id in cached[replica] else prefix_len) # Step 3
        cached[replica].add(prefix_id) # Step 4 · update state
        total += cost
    return total # Step 5

requests = [(i % 3, 1000, 50) for i in range(12)]
total_prefill_tokens(requests, 4, "round_robin") # 12,600 (12 prompt prefills)
total_prefill_tokens(requests, 4, "prefix") # 3,600 (3 prompt prefills)
```

Result: prefix-aware routing prefills **3.5× fewer** tokens.

Caveat (interview follow-up): pure prefix routing left replica 3 idle while 0–2 took all the load. Real inference-aware routers combine both: send to the replica holding the prefix unless it's overloaded, otherwise the least-loaded one.